

SOFTWARE REQUIREMENTS SPECIFICATION

Kisan-Setu: Low-Cost Agri-Logistics Aggregator

Prepared by: Leeladhar Bareth (RU-25-10750) | Guide: Mr. Ashutosh Banerjee | Date: 21 May 2026
School of Information Technology, Rungta International Skills University, Bhilai | MCA with Google | Session 2025-26

1. Introduction

Kisan-Setu (Farmer's Bridge) is a zero-commission, mobile-first vehicle booking web portal that directly connects rural Indian farmers with crop transporters, eliminating broker dependency. Built with React 18, Vite 5, Tailwind CSS v3, and Context API, it features two role-based dashboards — Farmer and Transporter — with a bilingual (Hindi/English) interface and 48px+ touch targets throughout.

This SRS defines the functional and non-functional requirements, system scope, technical stack, and constraints for the Kisan-Setu frontend (v1.0). The app uses mock data and is fully architected for backend integration.

2. Purpose & Scope

2.1 Purpose

To provide a simple, accessible browser-based platform enabling Indian farmers to arrange crop transport to the nearest mandi without brokers or hidden fees — demonstrating a scalable, role-based logistics frontend with zero backend dependency in its prototype phase.

2.2 In Scope

- Full React frontend with mock data — Farmer and Transporter dashboards.
- Farmer workflow: crop picker, multi-step booking form, mandi dropdown, transporter matching drawer, history.
- Transporter workflow: duty toggle, live job feed, accept/reject trips, earnings summary.
- Bilingual UI (Hindi/English), mobile-first (375px), Context API state management.

2.3 Out of Scope

- Real backend API, user authentication, GPS tracking, payments, notifications, admin dashboard.

3. Requirements

3.1 Functional Requirements

ID	Requirement
FR-01	Display a Home page with features overview, how-it-works section, and role-selection buttons (Farmer / Transporter).
FR-02	Farmer Dashboard shall have 4 tabs: Overview, Upcoming Trips, History, Nearby Transporters.
FR-03	Book Trip page shall provide a multi-step form: Crop Details → Location → Confirm.
FR-04	On booking, display a transporter matching drawer showing ETA, rating, vehicle type, and fare.
FR-05	Transporter Dashboard shall have 4 tabs: Requests, Active Trips, Earnings, History.
FR-06	Transporter dashboard shall include a duty-toggle (Online/Offline); live job feed activates only when Online.
FR-07	Transporter shall Accept or Reject each request; accepted trips move to Active Trips.
FR-08	A language toggle (Hindi/English) shall be accessible from the Navbar on all pages.
FR-09	Non-image files and invalid inputs shall trigger descriptive user-facing error messages.
FR-10	All interactive elements shall have a minimum 48px touch target height for mobile accessibility.

3.2 Non-Functional Requirements

ID	Requirement
NFR-01	UI shall remain non-blocking and responsive at all times using React's async state update model.
NFR-02	All page transitions and tab switches shall complete within 300ms on mid-range devices.
NFR-03	Application shall run on Chrome 90+, Firefox 88+, Edge 90+, and Safari 15+.
NFR-04	Mock data layer (mockData.js) shall be fully decoupled from UI — enabling API swap with zero UI changes.
NFR-05	Application shall be deployable as a static site on Vercel/Netlify via a single npm run build command.

4. Software & Hardware Requirements

4.1 Software Requirements

Item	Technology / Tool	Details
Frontend Framework	React.js 18+	Component-based SPA; Hooks + Context API.
Build Tool	Vite 5 + Node.js 18+	Fast dev server; production build to /dist.
Styling	Tailwind CSS v3	Utility-first CSS; no custom CSS files needed.
State Management	React Context API	Redux alternative for medium-scale state (AppContext).
Routing	React Router DOM v6	Client-side routing for all 4 main pages.
Language	JavaScript ES2022+	Optional chaining, async/await, destructuring.
Font	Baloo 2 (Google Fonts)	Supports Devanagari (Hindi) + Latin (English) scripts.
Version Control	Git + GitHub	Feature-branch workflow; hosted on GitHub.
Deployment	Vercel / Netlify / GitHub Pages	Static deployment; npm install → npm run dev.
Browser Support	Chrome / Firefox / Edge / Safari (latest)	All major modern browsers.

4.2 Hardware Requirements

Component	Development (Minimum)	End-User Device (Minimum)
Processor	Dual-core 2 GHz+	Any modern smartphone / desktop CPU
RAM	4 GB (8 GB recommended)	2 GB mobile / 4 GB desktop
Storage	200 MB free (project + node_modules)	No local storage needed (web app)
Display	1280×720 development viewport	375px mobile / 1024×768 desktop
Internet	Required for npm install + CDN fonts	Required for page load

5. Constraints & Assumptions

5.1 Constraints

C-01	Frontend-Only Architecture	No backend server or database in v1.0. All data served from mockData.js.
C-02	No Real Authentication	Role selection is simulated via navigation; no JWT or session management.
C-03	No Payment Processing	Fare display is reference only; no UPI/Razorpay gateway is connected.
C-04	No GPS / Real-Time Location	Vehicle tracking and ETAs are mock values; no live location API is used.
C-05	Hardcoded Translation Map	Bilingual support uses a flat JS map; complex pluralisation not supported without react-i18next.
C-06	Browser Compatibility	Tested on Chrome 90+, Firefox 88+, Edge 90+, Safari 15+ only. Older browsers unsupported.

5.2 Assumptions

- Users have access to a modern smartphone or desktop browser with a stable internet connection for initial load.
- Farmers are assumed to have basic smartphone literacy; UI minimises text input, relying on tap-based selection.
- Node.js 18+ and npm 9+ are installed on the developer's machine for local setup.
- Deployment targets a static hosting provider (Vercel/Netlify); no SSR required for v1.0.
- All crop types, mandi names, and transporter data shown are representative mock values for academic demonstration.
- Future backend integration will follow RESTful API design; mock data is already structured to mirror expected API response shapes.

Zero Commission • Direct Mandi Rates • Kisan-Setu